

BANK ACCOUNT MANAGEMENT SYSTEM

Mini Project Documentation

Java Programming Project

Prepared by: Dharmit Sathvara

Roll No.:150096725080

Course : Java

1. Abstract

The Bank Account Management System is a Java-based desktop application designed to manage basic bank account operations through a graphical user interface. The system represents customers and accounts using object-oriented classes and provides functions for creating, searching, updating and deleting accounts, as well as depositing and withdrawing money. It also records deposit and withdrawal transactions and supports saving and reading transaction information from a text file.

The project demonstrates the use of Java classes and objects, encapsulation, constructors, arrays, LinkedList, HashMap, HashSet, TreeMap, file handling, exception handling, Java Swing and event handling. An Account object contains a Customer object, while AccountArray demonstrates array-based storage of multiple accounts. BankManager combines different data structures for duplicate checking, key-based account retrieval and sorted display. TransactionManager maintains a dynamic transaction list and handles file operations.

The expected outcome is a functional desktop banking-management prototype that organizes account information, performs common account operations, provides user feedback for invalid input, and demonstrates how multiple Java concepts can be combined in one practical application.

2. Introduction

Managing customer and bank account information manually can become difficult when the number of accounts and transactions increases. A simple management application can organize account information and provide a consistent way to create, search, update and delete records. It can also reduce repetitive manual work for basic account operations.

2.1 Background of the Problem

A bank account record contains several related pieces of information such as account number, account type, customer name, phone number and balance. In addition, deposits and withdrawals create transaction records that need to be maintained. A useful application therefore needs both account management and transaction management functions.

2.2 Existing Challenges

- Maintaining customer and account information in an organized form.
- Finding an account quickly using its account number.
- Preventing duplicate account numbers.
- Maintaining a list of deposit and withdrawal transactions.
- Handling invalid numeric input and operations such as withdrawal beyond the available balance.
- Providing a simple interface through which a user can perform operations.

2.3 Need for the Proposed System

The proposed system combines object-oriented design, data structures, file handling and a Swing graphical interface in one application. Instead of keeping account information as unrelated variables, the system models customers and accounts as objects. It uses different Java collections according to the purpose of the data, such as HashMap for account lookup and TreeMap for sorted display.

2.4 Proposed Solution

The application provides a Swing-based interface. User input is processed through ActionListener events. BankManager performs account management, Account performs balance operations, AccountArray demonstrates array storage, and TransactionManager maintains and stores transaction records. Error conditions are handled through validation and exception handling.

2.5 Scope

The current project is a desktop prototype for basic account and transaction management. It supports account creation, search, update, deletion, deposit, withdrawal, transaction viewing, transaction file saving/reading and display of all accounts. It is not presented as a full banking platform: the code does not implement a database, online services, multi-user authentication, encryption, or production banking security.

3. Problem Statement

The project addresses the problem of managing basic customer, bank-account and transaction information in a small desktop application. The intended user is a person operating the account-management application. The user needs to create accounts, locate existing accounts, modify customer/account information, delete accounts, perform deposits and withdrawals, and view transaction information. The proposed Java application solves this by combining a Swing interface with object-oriented classes and suitable data structures for storage, retrieval, sorting and transaction management.

4. Objectives

- To develop a Java-based Bank Account Management application.
- To represent customers, accounts and transactions using classes and objects.
- To demonstrate encapsulation through private fields and controlled access methods.
- To use an array for storing multiple Account objects.
- To use LinkedList for dynamically maintaining transactions.
- To use HashMap for key-based account searching and retrieval.
- To use HashSet to prevent duplicate account numbers.
- To use TreeMap to maintain accounts in sorted key order.
- To implement create, search, update and delete account operations.
- To implement deposit and withdrawal operations with balance checking.
- To develop a user-friendly Java Swing interface.
- To implement input validation and exception handling.
- To provide file saving and reading for transaction information.

5. Scope of the Project

Area	Implemented in the project
Target users	Operator/user of the desktop Bank Account Management System
Account management	Create, search, update, delete and display accounts
Banking operations	Deposit and withdrawal with balance check
Customer information	Name and phone number
Account information	Account number, account type and balance

Transactions	Record deposit and withdrawal type and amount
Data structures	Array, LinkedList, HashMap, HashSet and TreeMap
File handling	Save and read transactions.txt
Interface	Java Swing GUI
Limitations	Desktop-only prototype; no database, login/authentication or online banking

6. Concepts Used

The following sections explain the actual role of each Java concept in this project rather than giving only textbook definitions.

6.1 Classes and Objects

Customer represents the account holder and stores name and phone. Account represents a bank account and stores account number, account type, balance and a Customer object. Transaction represents a transaction with a type and amount. BankManager and TransactionManager provide application-level operations. Objects are created when the system creates customers, accounts and transactions.

6.2 Encapsulation

Customer and Account keep their main fields private. Access is provided through getters and selected setters and methods. For example, Account does not expose a public balance variable; balance is changed through deposit() and withdraw(). This keeps balance modification under the rules implemented by the class.

6.3 Constructors

Constructors initialize Customer, Account and Transaction objects. For example, an Account constructor receives the account number, account type, starting balance and Customer object and assigns them to the new account.

6.4 Arrays

AccountArray contains an Account[] array. The array stores multiple Account objects, while count tracks how many positions are currently used. The add operation checks capacity before inserting an account. The array therefore demonstrates fixed-size indexed storage of account objects.

6.5 LinkedList

TransactionManager uses LinkedList<Transaction> to maintain transaction records. Every deposit or successful withdrawal creates a Transaction object and adds it to the list. This is suitable for a collection whose number of records can change during program execution.

6.6 HashMap

BankManager uses HashMap<Integer, Account>, where the account number is the key and the Account object is the value. searchAccount(accountNumber) retrieves the account using that key.

6.7 HashSet

BankManager uses HashSet<Integer> to keep account numbers unique. Before an account is added, the program checks whether the number is already present. If it exists, the new account is rejected.

6.8 TreeMap

BankManager also stores accounts in TreeMap<Integer, Account>. The account number is the key. The TreeMap is used when displaying all accounts so that the accounts are obtained in key-sorted order.

6.9 Swing

The main class extends JFrame and creates Swing components including JLabel, JTextField, JComboBox, JButton and JOptionPane. These components provide the graphical user interface for entering account details and triggering operations.

6.10 Event Handling

The main GUI class implements ActionListener. Each button is registered with the listener, and actionPerformed() checks which button generated the event. The corresponding operation is then executed, such as create, search, deposit, withdrawal, update or delete.

6.11 File Handling

TransactionManager uses FileWriter to save transaction text to transactions.txt and Scanner to read the file. IOException and FileNotFoundException are handled so that file problems do not terminate the application unexpectedly.

6.12 Exception Handling and Validation

The GUI uses try-catch blocks to handle invalid numeric input. It also validates empty customer fields during account creation, positive amounts for deposits and withdrawals, duplicate account numbers, missing accounts and insufficient balance.

7. System Requirements and Functionalities

7.1 Functionalities

1. Create Account
2. Search Account
3. Deposit Money
4. Withdraw Money
5. Update Account
6. Delete Account
7. View Transactions
8. Save Transactions to File
9. Read Transactions from File
10. Display All Accounts

7.2 Basic Hardware/Software Requirements

Requirement	Suggested/used environment
Programming language	Java
GUI toolkit	Java Swing / AWT event classes
Development environment	Any Java-compatible IDE or command-line environment
Storage	Local text file: transactions.txt
Operating system	Desktop operating system capable of running Java

8. System Design

8.1 System Architecture

```
User
|
v
Swing GUI
|
v
Event Handling (ActionListener)
|
+--> BankManager
|     +--> HashMap
|     +--> HashSet
|     +--> TreeMap
|     +--> AccountArray
|
+--> Account / Customer
|
+--> TransactionManager
|     +--> LinkedList
|     +--> transactions.txt
|
v
Output / JOptionPane
```

The architecture follows the documentation guideline's User → Swing GUI → Application Logic → Data Structures → Output flow. The exact classes in the implementation provide the application-logic and data-structure layers.

8.2 Class Design / UML-style Representation

```
Customer
- name : String
- phone : String
+ getName()
+ getPhone()
+ setName()
+ setPhone()

1
|
| has-a
v
Account
- accountNumber : int
- accountType : String
- balance : double
- customer : Customer
+ getAccountNumber()
+ getAccountType()
+ getBalance()
+ getCustomer()
+ setAccountType()
+ deposit()
+ withdraw()
+ getAccountDetails()
```

```

AccountArray
- accounts : Account[]
- count : int
+ add()
+ getAccounts()

Transaction
- type : String
- amount : double
+ toString()

TransactionManager
- transactions : LinkedList<Transaction>
+ addTransaction()
+ getTransactions()
+ saveToFile()
+ readFromFile()

BankManager
- accounts : HashMap<Integer, Account>
- accountNumbers : HashSet<Integer>
- sortedAccounts : TreeMap<Integer, Account>
- accountArray : AccountArray
+ addAccount()
+ searchAccount()
+ updateAccount()
+ deleteAccount()
+ getAllAccounts()

BankAccountManagementSystem
extends JFrame
implements ActionListener
+ actionPerformed()
+ main()

```

8.3 Data Flow

```

User enters account details
|
v
Swing fields / buttons
|
v
actionPerformed()
|
+--> Create --> Customer + Account --> BankManager
|
+--> Search --> HashMap --> Account
|
+--> Deposit/Withdraw --> Account --> TransactionManager
|
+--> Update/Delete --> BankManager
|
+--> Save/Read --> transactions.txt
|
v
JOptionPane output

```

8.4 Flowchart – Create Account

```
START
|
v
Enter account number, name, phone and type
|
v
Convert account number to integer
|
v
Create Customer object
|
v
Create Account object
|
v
Check account number in HashSet
|
+---- Already exists ----> Show error
|
v
Add account to HashMap, HashSet, TreeMap and AccountArray
|
v
Show success message
|
v
END
```

9. Data Structure Design

Data Structure	Actual purpose in project	Reason for selection
Account[] Array	Stores Account objects in AccountArray	Demonstrates fixed-size indexed storage
LinkedList<Transaction>	Maintains deposit and withdrawal transaction records	Supports a changing list of transactions
HashMap<Integer,Account>	Maps account number to Account object for search	Key-based account retrieval
HashSet<Integer>	Stores account numbers for duplicate checking	Does not allow duplicate account numbers
TreeMap<Integer,Account>	Stores accounts by account number for sorted display	Maintains keys in sorted order

10. Module Description

Module 1 – Customer and Account / OOP

Customer stores customer details. Account stores account information, links the customer object, and performs deposit and withdrawal operations. AccountArray demonstrates array-based account storage.

Module 2 – Transactions and File Handling

Transaction represents a transaction type and amount. TransactionManager stores transactions in a LinkedList, displays them, writes them to transactions.txt and reads them back.

Module 3 – Account Management and Data Structures

BankManager coordinates the HashMap, HashSet, TreeMap and AccountArray. It adds accounts after duplicate checking, searches by account number, updates customer/account details, deletes accounts and displays sorted accounts.

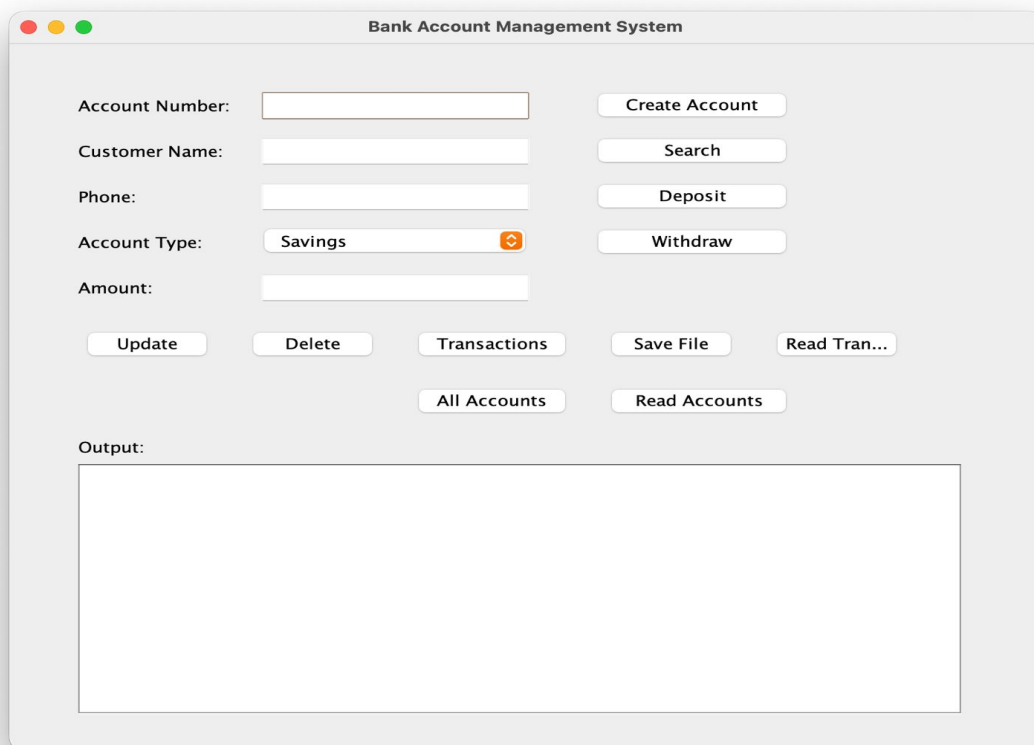
Module 4 – Swing GUI and Event Handling

BankAccountManagementSystem creates the Swing window and controls. ActionListener connects button clicks to the application operations. JOptionPane is used for success, error and result messages.

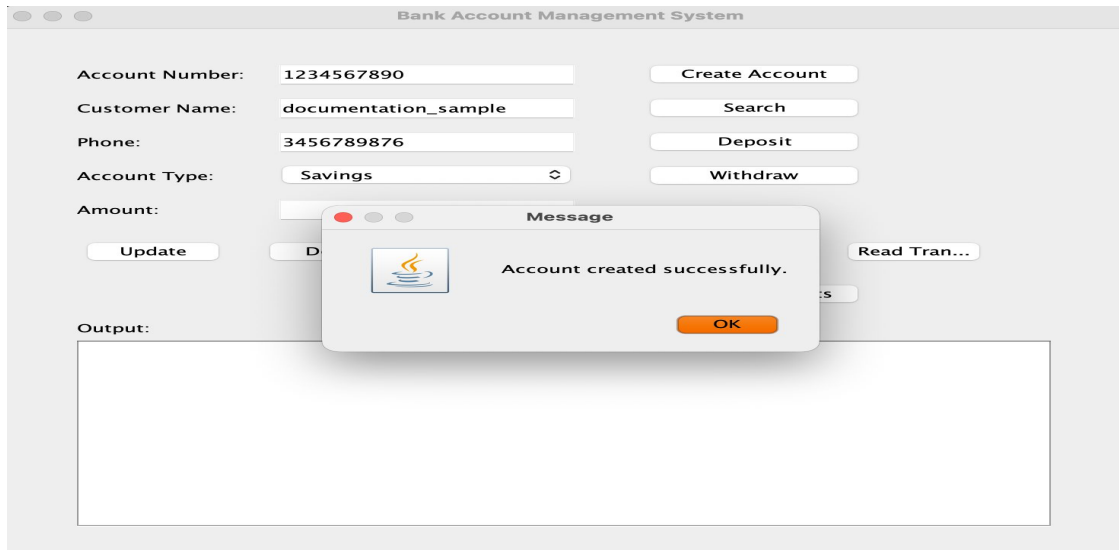
11. User Interface Design

The application uses a Swing window with fields for account number, customer name, phone, amount and account type. Buttons are provided for account creation, search, deposit, withdrawal, update, delete, transactions, saving, reading and displaying all accounts.

Actual application screenshots should be inserted in this section. The documentation guidelines require clear screenshots with captions; screenshots were not included with the supplied source/documentation files, so no screenshots are fabricated here. **Main Window**



Account Creation And Validation



The screenshot shows the 'Bank Account Management System' window. The 'Account Number' field contains '1234567890', 'Customer Name' contains 'documentation_sample', 'Phone' contains '3456789876', and 'Account Type' is set to 'Savings'. A 'Message' dialog box is displayed in the center, stating 'Account created successfully.' with an 'OK' button. The 'Amount' field is empty, and the 'Output' area is a large empty text box.

Bank Account Management System

Account Number: 1234567890 Create Account

Customer Name: documentation_sample Search

Phone: 3456789876 Deposit

Account Type: Savings Withdraw

Amount: Update

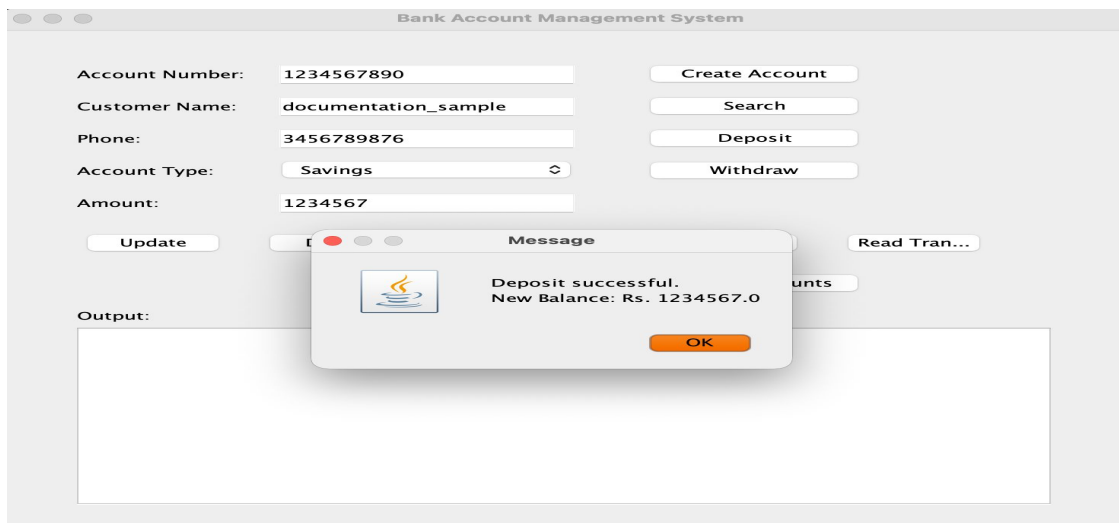
Message

Account created successfully.

OK

Output:

Deposit Result



The screenshot shows the 'Bank Account Management System' window. The 'Account Number' field contains '1234567890', 'Customer Name' contains 'documentation_sample', 'Phone' contains '3456789876', and 'Account Type' is set to 'Savings'. The 'Amount' field now contains '1234567'. A 'Message' dialog box is displayed in the center, stating 'Deposit successful. New Balance: Rs. 1234567.0' with an 'OK' button. The 'Output' area is a large empty text box.

Bank Account Management System

Account Number: 1234567890 Create Account

Customer Name: documentation_sample Search

Phone: 3456789876 Deposit

Account Type: Savings Withdraw

Amount: 1234567 Update

Message

Deposit successful.
New Balance: Rs. 1234567.0

OK

Output:

Withdraw Result

Bank Account Management System

Account Number:

Customer Name:

Phone:

Account Type:

Amount:

Output:

Withdrawal successful.
Remaining Balance: Rs. 1233333.0

OK

Save File

Bank Account Management System

Account Number:

Customer Name:

Phone:

Account Type:

Amount:

Output:

New transactions appended to transactions.txt
Accounts saved to accounts.txt

OK

Account Search

Bank Account Management System

Account Number:

Customer Name:

Phone:

Account Type:

Amount:

Output:

Message

 Account Number: 1234567890
Customer Name: documentation_sample
Phone: 3456789876
Account Type: Savings
Balance: Rs. 1233333.0

Account Deleted

Bank Account Management System

Account Number:

Customer Name:

Phone:

Account Type:

Amount:

Output:

Message

 Account deleted successfully.

1234,asdf,1234,Savings,123333.0
12341234,asdf123,123412345,Savings,123123.0
1234567890,documentation_sample,3456789876,Savings,1233333.0

Account Update

Bank Account Management System

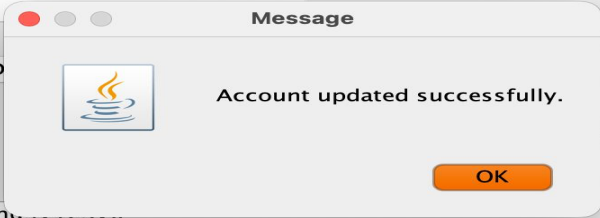
Account Number:

Customer Name:

Phone:

Account Type:

Amount:



Message

 Account updated successfully.

Output:

```
1234,asdf,1234,Savings,123.0
12341234,asdf123,123412345,Savings,123123.0
1234567890,documentation_sample,3456789876,Savings,1233333.0
```

Read Accounts

Bank Account Management System

Account Number:

Customer Name:

Phone:

Account Type:

Amount:

Output:

```
1234,asdf,1234,Savings,123.0
12341234,asdf123,123412345,Savings,123123.0
1234567890,documentation_sample,3456789876,Savings,1233333.0
```

Transaction History

Bank Account Management System

Account Number: 1234567890

Create Account

Customer Name: documentation_sample

Search

Phone: 3456789876

Deposit

Account Type: Savings

Withdraw

Amount:

Update

Delete

Transactions

Save File

Read Tran...

All Accounts

Read Accounts

Output:

TYPE	AMOUNT (Rs.)
Deposit	34567.00
Withdrawal	3457.00
Deposit	123.00
Deposit	123123.00
Deposit	123123.00
Deposit	1234567.00
Withdrawal	1234.00

All Accounts Display

Bank Account Management System

Account Number: 1234567890

Create Account

Customer Name: documentation_sample

Search

Phone: 3456789876

Deposit

Account Type: Savings

Withdraw

Amount:

Update

Delete

Transactions

Save File

Read Tran...

All Accounts

Read Accounts

Output:

ACC NO	CUSTOMER	PHONE	TYPE	BALANCE
1234	asdf	1234	Savings	123.00
12341234	asdf123	123412345	Savings	123123.00
1234567890	documentation_sample	3456789876	Savings	1233333.00

Figure	Screenshot to insert	Caption
Figure 1	Main application window	Main Window of Bank Account Management System
Figure 2	Account creation interaction	Account Creation and Validation
Figure 3	Search result	Account Search Result
Figure 4	Deposit/withdrawal result	Deposit or Withdrawal Result
Figure 5	Transactions display	Transaction History
Figure 6	All accounts display	All Accounts Display
Figure 7	Error message	Validation / Error Message

12. Implementation

12.1 Customer and Account Implementation

Customer and Account are implemented as separate classes. Customer uses private fields with getters and setters. Account uses a Customer reference, account fields, a constructor, deposit(), withdraw(), and getAccountDetails().

12.2 Account Array Implementation

AccountArray creates an Account[] based on the requested size. add() checks whether the array has capacity, stores the account at the current count position and increments count.

12.3 LinkedList and Transaction Implementation

TransactionManager creates a LinkedList<Transaction>. addTransaction() creates a Transaction and adds it to the list. getTransactions() loops through the list and builds a display string.

12.4 HashMap, HashSet and TreeMap Implementation

BankManager maintains three complementary collections. HashSet is used to reject duplicate account numbers, HashMap stores accounts for key-based search, and TreeMap stores accounts for sorted display. The same Account object is placed into the relevant structures when an account is added.

12.5 CRUD Operations

Operation	Implementation
Create	Create Customer and Account objects and add the account through BankManager
Read	Search an account using HashMap and display account details; display all accounts using TreeMap
Update	Find the account and change customer name, phone and account type
Delete	Remove the account from HashMap, HashSet and TreeMap

12.6 Swing Components and Event Handling

The GUI extends JFrame and implements ActionListener. Buttons are connected with addActionListener(this). actionPerformed() determines the source button and executes the appropriate operation.

12.7 File Handling

The Save File action calls `saveToFile()`, which writes each transaction to `transactions.txt`. The Read File action calls `readFromFile()`, which reads the file line by line.

12.8 Exception Handling

try-catch blocks surround operations that can receive invalid input or file errors. `NumberFormatException` is handled for invalid account numbers, while broader input exceptions are also caught in the GUI. `FileWriter` and `Scanner` operations handle `IOException` and `FileNotFoundException` in `TransactionManager`.

13. Exception Handling and Validation

Situation	Handling in the project	User response
Empty name or phone during creation	Checked before account creation	Please enter all customer details.
Invalid account number	<code>Integer.parseInt()</code> is inside try-catch	Enter a valid account number.
Duplicate account number	HashSet check in <code>BankManager</code>	Account number already exists.
Invalid/non-positive deposit	Deposit amount is checked	Amount must be greater than zero.
Account not found	Search result is checked for null	Account not found.
Insufficient balance	<code>withdraw()</code> returns false	Insufficient balance.
Invalid general input	GUI catch block	Enter valid information / Invalid input.
File does not exist	<code>FileNotFoundException</code> handled	Transaction file not found.
File write error	<code>IOException</code> handled	Error while writing file.

14. Results and Discussion

The implemented application combines the four project areas into one desktop system. The source provides account creation, account search, deposit, withdrawal, update, delete, transaction display, transaction file saving/reading and sorted account display. The data structures are used for different responsibilities instead of using a single collection for every task.

14.1 Searching and Sorting

Account search is performed through the `HashMap` using the account number as the key. Account display uses the `TreeMap` values, which provides account-number key ordering. The `HashSet` separately supports the duplicate-account-number check.

14.2 Effectiveness of Data Structures

Each structure has a distinct role: the array demonstrates indexed fixed-size account storage, `LinkedList` maintains a changing transaction list, `HashMap` supports key-based retrieval, `HashSet` enforces uniqueness, and `TreeMap` provides ordered keys. This separation makes the project useful for demonstrating the Java data structures taught in the course.

14.3 GUI Experience

The Swing interface places the major banking operations behind buttons and uses dialog messages for results and errors. This provides a direct desktop interaction model without requiring command-line input.

14.4 Development Issues

- Handling invalid numeric input from text fields.
- Preventing duplicate account numbers.
- Checking account existence before operations.
- Preventing withdrawals when the balance is insufficient.
- Handling missing or inaccessible transaction files.
- Connecting multiple GUI buttons to a single ActionListener.

These issues are addressed using validation, conditional checks, try-catch blocks, HashSet duplicate checking and the Account.withdraw() balance check.

15. Advantages

- Simple desktop interface for basic account operations.
- Organized separation of customer, account, transaction and management responsibilities.
- Fast key-based account lookup through HashMap.
- Sorted account display through TreeMap.
- Duplicate account numbers are checked using HashSet.
- Transactions are maintained separately using LinkedList.
- Basic transaction persistence is provided through a text file.
- Invalid input and common operation errors produce user messages.

16. Limitations

- The project is a desktop application and does not provide a web or mobile interface.
- Account data is maintained in memory during application execution; the implemented file handling is specifically for transactions.
- No database such as MySQL or MongoDB is used.
- No login, authentication or role-based access control is implemented.
- No encryption or production-level banking security is implemented.
- The transaction text file is a simple local storage mechanism rather than a database.
- The current GUI is intended as a mini-project prototype rather than a complete banking platform.

17. Future Enhancements

- Connect the system to a relational database for persistent account and transaction storage.
- Add user login and authentication.
- Add role-based access such as administrator and operator.
- Add stronger input validation for phone numbers and other fields.
- Add transaction dates, account-specific transaction history and transaction IDs.
- Provide data export and report generation.
- Develop a web-based or mobile version.
- Add stronger security controls and encrypted storage.
- Add automated tests for account and transaction operations.

18. Conclusion

The Bank Account Management System was developed as a Java desktop application to demonstrate how object-oriented programming, data structures, file handling, exception handling and Swing can be combined to solve a practical management problem. The system models customers, accounts and transactions as objects and provides common operations such as account creation, searching, updating, deletion, deposits and withdrawals.

The project applies encapsulation through private fields and controlled methods, uses an Account array for fixed-size storage, LinkedList for transactions, HashMap for key-based retrieval, HashSet for duplicate account-number checking and TreeMap for sorted account display. Swing and ActionListener provide the graphical interface and event-driven interaction, while file handling provides local storage for transaction records. Validation and exception handling improve the behavior when users provide invalid input or when an account or file is unavailable.

The project provides a useful learning model for a small account-management application. It also shows how the same business entity can be represented and managed through different Java data structures according to the required operation. Future versions can extend the prototype with database storage, authentication, stronger security and web or mobile interfaces.

19. References

The following references should be finalized according to the resources actually used by the project team:

- Java SE API documentation – classes and interfaces used in the project.
- Java Swing and AWT event-handling documentation.
- Course notes / lecture material for Java Programming and data structures.
- Any tutorials or learning resources actually consulted during development.

Note: The supplied documentation guideline requires a consistent reference format. The project team should replace this section with the exact resources and URLs/books they actually used.

Appendix A – Selected Important Code Sections

The documentation guideline recommends not placing the complete source code in the main documentation. The following small sections can be included in an appendix if required by the instructor.

A.1 Encapsulation and Account Operations

```
private double balance;

public void deposit(double amount) {
    balance = balance + amount;
}

public boolean withdraw(double amount) {
    if (amount <= balance) {
        balance = balance - amount;
        return true;
    }
}
```

```
        return false;
    }
}
```

A.2 HashMap and Duplicate Checking

```
int number = account.getAccountNumber();

if (accountNumbers.contains(number)) {
    return false;
}

accountNumbers.add(number);
accounts.put(number, account);
sortedAccounts.put(number, account);
```

A.3 LinkedList Transaction Storage

```
private LinkedList<Transaction> transactions;

public void addTransaction(String type, double amount) {
    transactions.add(new Transaction(type, amount));
}
}
```

A.4 File Saving

```
FileWriter writer =
    new FileWriter("transactions.txt");

for (Transaction transaction : transactions) {
    writer.write(transaction.toString() + "\n");
}
}
```

A.5 Swing Event Handling

```
public class BankAccountManagementSystem
    extends JFrame implements ActionListener {

    public void actionPerformed(ActionEvent e) {
        // execute operation based on clicked button
    }
}
```